Attention

The Smol Training Playbook

While feedforward layers dominate compute during pretraining, attention becomes the main bottleneck at inference (especially with long contexts), where it drives up compute cost and GPU memory requirements, reducing throughput.

Table 1 — Training (forward pass, per layer, batch = 1, sequence L)

Component	FLOPs	HBM bytes moved	Arithmetic intensity
FFN (up + down)	$16 \cdot L \cdot d^2$	$16 \cdot d^2$ (weights)	$\sim L$
Attn projections (Q, K, V, O)	$8 \cdot L \cdot d^2$	$8 \cdot d^2$ (weights)	$\sim L$
Attention scores (QK^T + ·V)	$4 \cdot L^2 \cdot d$	$\sim 8 \cdot L \cdot d$ (Q, K, V acts)	$\sim L/2$
Per-layer total	$24 \cdot L \cdot d^2 + 4 \cdot L^2 \cdot d$	$24 \cdot d^2 + 8 \cdot L \cdot d$	hundreds → **compute-bound**

Table 2 — Decode (per generated token, per layer, batch = 1)

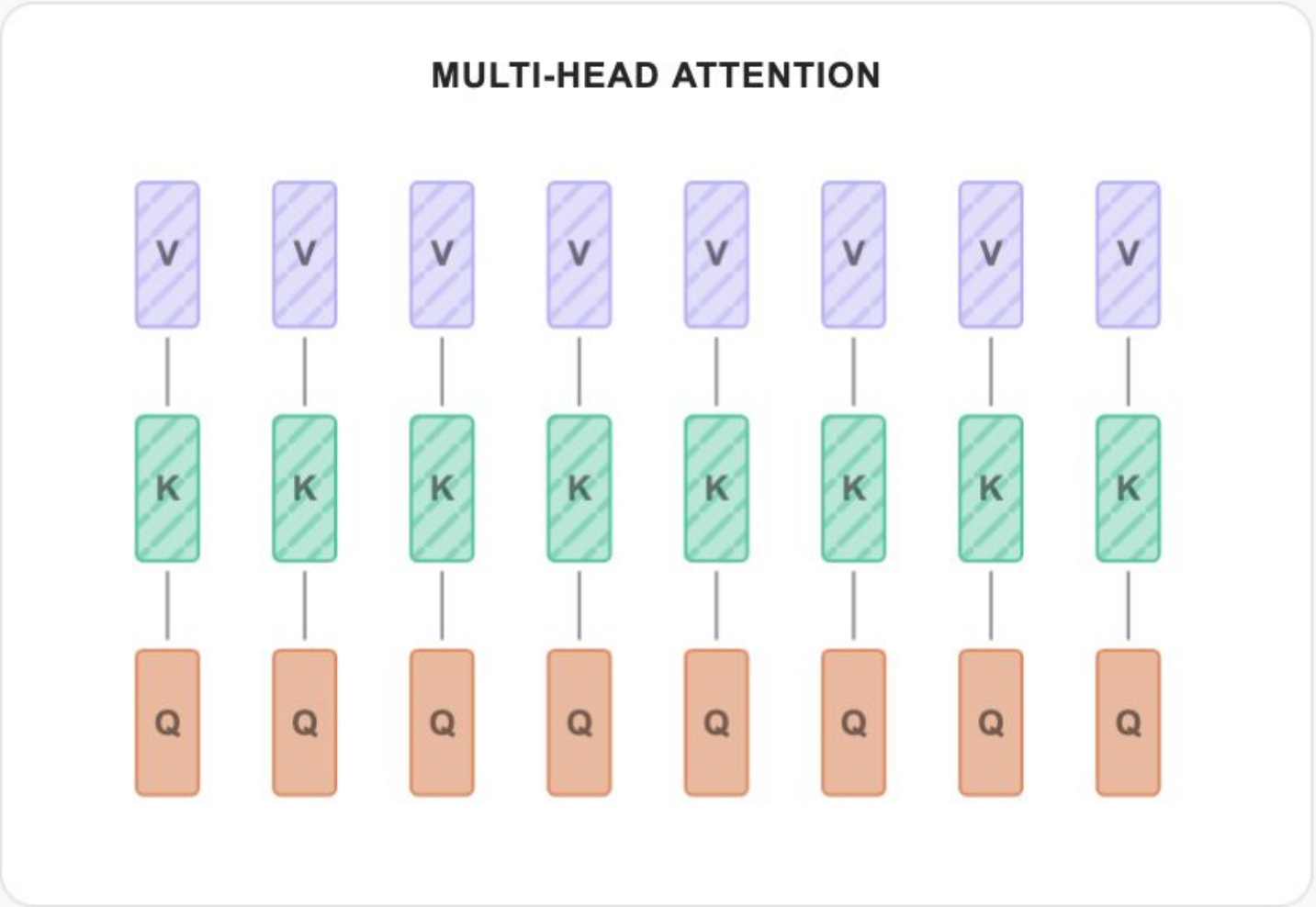
Component	FLOPs	HBM bytes read	Arithmetic intensity
FFN (up + down)	$16 \cdot d^2$	$16 \cdot d^2$ (weights)	~ 1
Attn projections (Q, K, V, O)	$8 \cdot d^2$	$8 \cdot d^2$ (weights)	~ 1
Attention over KV cache	$4 \cdot L \cdot d$	$4 \cdot L \cdot d$ (KV cache)	~ 1
Per-layer total	$24 \cdot d^2 + 4 \cdot L \cdot d$	$24 \cdot d^2 + 4 \cdot L \cdot d$	**≈ 1 → memory-bound**

1. Bulk pretraining at short L
2. Long-context extension phase
3. SFT / RLHF on long contexts

1. Bulk pretraining at short L
2. Long-context extension phase
3. SFT / RLHF on long contexts

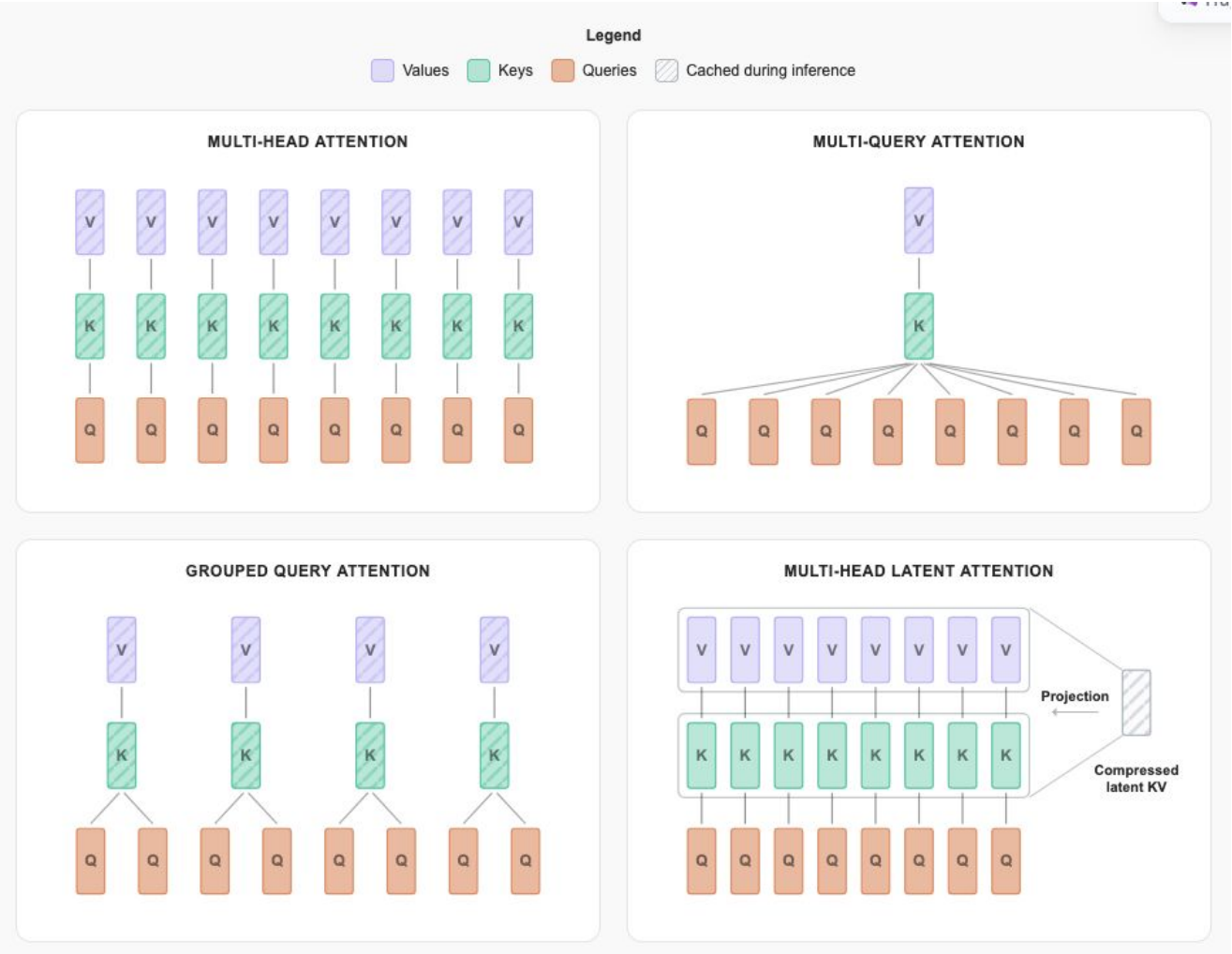
1. Llama 3: 8K -> 128K
2. Qwen2.5: 4K -> 32K, 32K -> 64K -> 128K -> 256K
3. Qwen3: 4K -> ??? (knowledge intensive data: STEM, coding) -> 32K
4. DeepSeek-V3: 4K -> 32K -> 128K
5. Yi: 4K -> 200K (1-2B tokens is enough)

Multi-head attention (MHA)


MULTI-HEAD ATTENTION
V
V
V
V
V
V
V
V
K
K
K
K
K
K
K
K
Q
Q
Q
Q
Q
Q
Q
Q

$$
\begin{aligned}
s_{KV} &= 2 \times n_{bytes} \times seq \times n_{layers} \times n_{heads} \times dim_{heads} \\
&= 2 \times 2 \times 8192 \times 32 \times 32 \times 128 = 4 \text{ GB } \textit{(Llama 3 8B)} \\
&= 2 \times 2 \times 8192 \times 80 \times 64 \times 128 = 20 \text{ GB } \textit{(Llama 3 70B)}
\end{aligned}
$$

70B, 1M context: 2621 GB


Legend
Values
Keys
Queries
Cached during inference
MULTI-HEAD ATTENTION
MULTI-QUERY ATTENTION
GROUPED QUERY ATTENTION
MULTI-HEAD LATENT ATTENTION
V
K
Q
Projection
Compressed latent KV

Step 1 — Down-project hidden state into KV latent

$$c_t^{KV} = W_{DKV}\, h_t$$

Symbol	What it is	Shape
h_t	hidden state (residual-stream vector) at token t, input to this attention layer	$\mathbb{R}^d = \mathbb{R}^{5120}$
W_{DKV}	learned down-projection matrix (one per layer, shared across all heads and across all tokens)	$\mathbb{R}^{d_c \times d} = \mathbb{R}^{512 \times 5120}$
c_t^{KV}	**KV latent** at token t — the compressed summary that will be cached	$\mathbb{R}^{d_c} = \mathbb{R}^{512}$

Step 2 — Define per-head K and V as linear readouts of the latent

$$k_t^{(i)} = W_{UK}^{(i)} c_t^{KV}, \qquad v_t^{(i)} = W_{UV}^{(i)} c_t^{KV}$$

Symbol	What it is	Shape
$W_{UK}^{(i)}$	up-projection for head i's key; learned, head-specific, fixed after training	$\mathbb{R}^{d_h \times d_c} = \mathbb{R}^{128 \times 512}$
$W_{UV}^{(i)}$	up-projection for head i's value	$\mathbb{R}^{d_h \times d_c} = \mathbb{R}^{128 \times 512}$
$k_t^{(i)}$	head-i key vector at token t — what standard MHA would cache	$\mathbb{R}^{d_h} = \mathbb{R}^{128}$
$v_t^{(i)}$	head-i value vector at token t	$\mathbb{R}^{d_h} = \mathbb{R}^{128}$

These are *definitions*, not computations you run at decode. Think of them as saying "if anyone asks, head i's key at token t is this function of the latent." The absorption trick in Step 4 means no one ever asks.

Equivalent factored view: the plain-MHA matrix $W_K^{(i)} \in \mathbb{R}^{d_h \times d}$ has been replaced by the product $W_{UK}^{(i)} W_{DKV}$. That's a rank-d_c approximation to $W_K^{(i)}$, shared across heads via the common right factor W_{DKV}.

Step 3 — Down-project hidden state into Q latent

$$c_t^Q = W_{DQ}\, h_t, \qquad q_t^{(i)} = W_{UQ}^{(i)}\, c_t^Q$$

Symbol	What it is	Shape
W_{DQ}	down-projection into the Q latent (per layer, shared across heads)	$\mathbb{R}^{d'_c \times d} = \mathbb{R}^{1536 \times 5120}$
c_t^Q	**Q latent** at current token t	$\mathbb{R}^{d'_c} = \mathbb{R}^{1536}$
$W_{UQ}^{(i)}$	up-projection into head i's query	$\mathbb{R}^{d_h \times d'_c} = \mathbb{R}^{128 \times 1536}$
$q_t^{(i)}$	head-i query at token t	$\mathbb{R}^{d_h} = \mathbb{R}^{128}$

Step 4 — Absorption: rewrite the attention score on latents

Starting from the standard dot-product score between current token t and past token s, for head i:

$$\text{score}_i(t, s) = q_t^{(i)\top} k_s^{(i)}$$

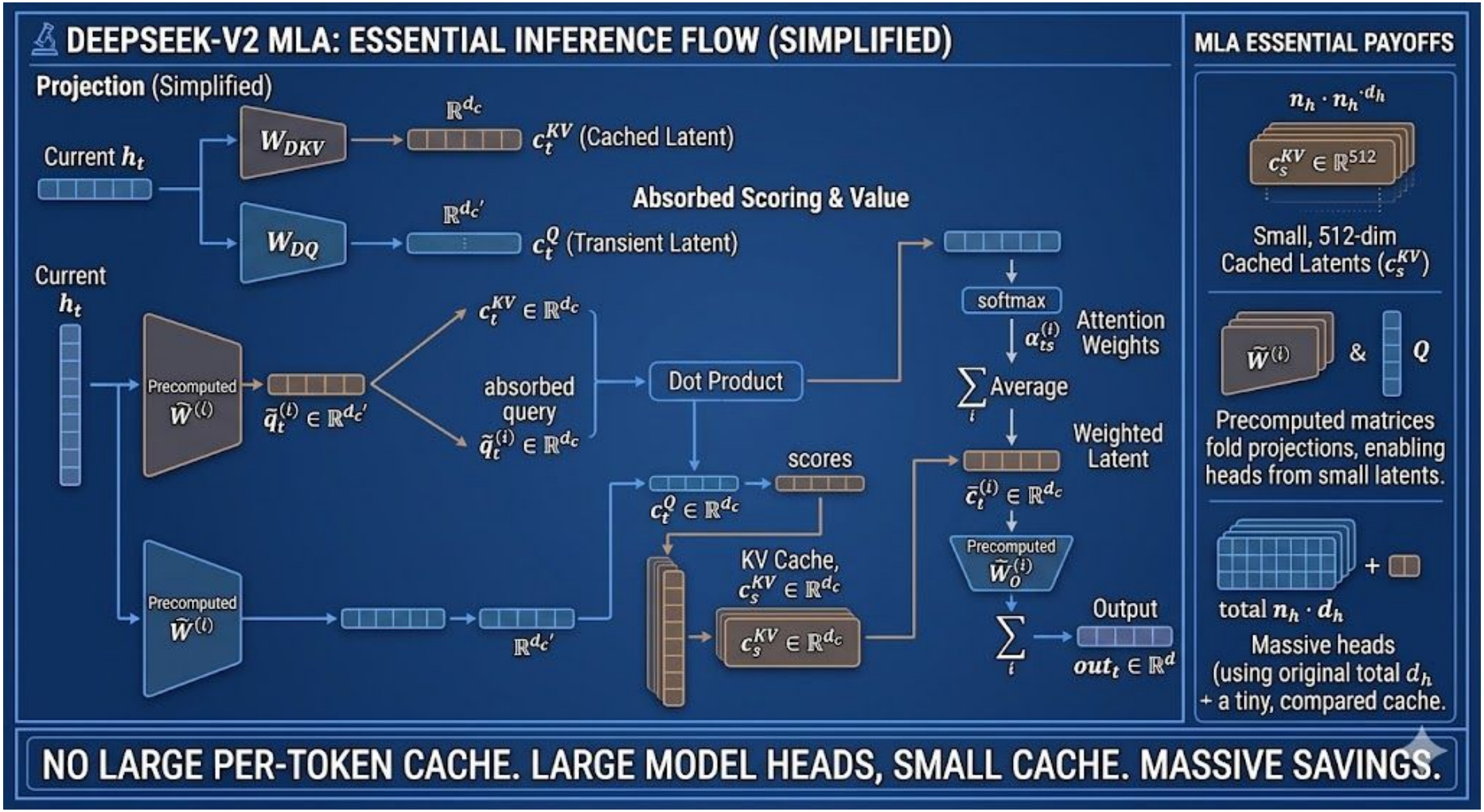
Substitute the definitions of q and k:

$$= \left(W_{UQ}^{(i)} c_t^Q\right)^\top \left(W_{UK}^{(i)} c_s^{KV}\right) = \left(c_t^Q\right)^\top \left(W_{UQ}^{(i)\top} W_{UK}^{(i)}\right) c_s^{KV}$$

Name the middle product:

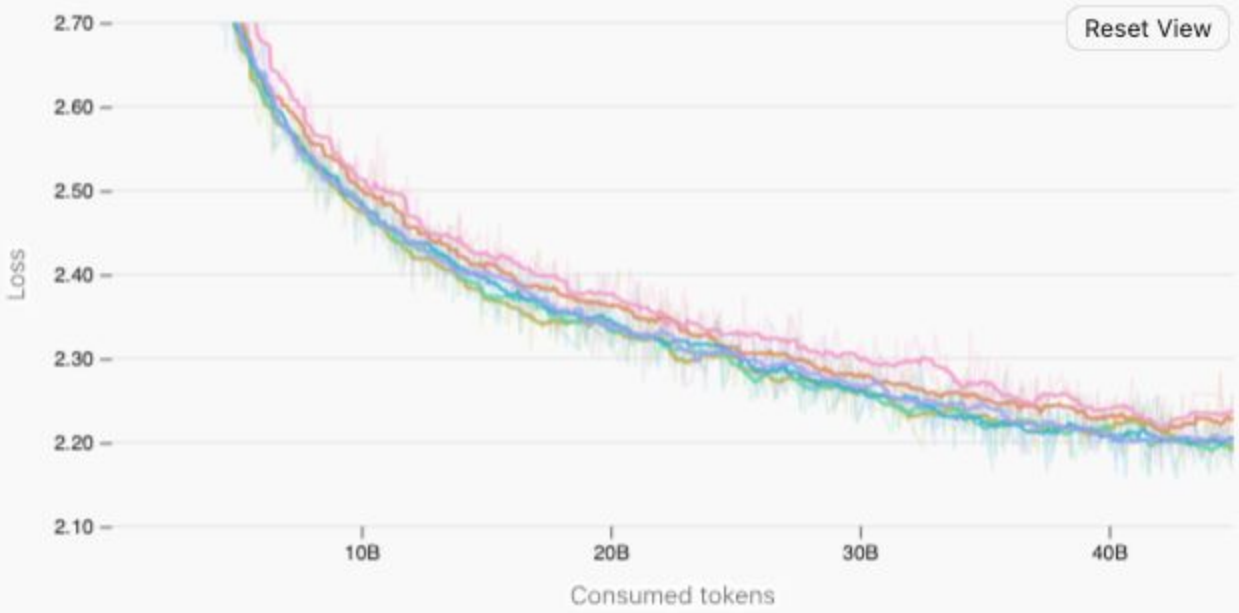
$$\tilde{W}^{(i)} := W_{UQ}^{(i)\top} W_{UK}^{(i)}$$

Symbol	What it is	Shape
$\tilde{W}^{(i)}$	**absorbed head matrix** for head i — a bilinear form that acts directly on latents	$\mathbb{R}^{d_c' \times d_c} = \mathbb{R}^{1536 \times 512}$
$\text{score}_i(t, s)$	pre-softmax attention logit (a scalar)	$\mathbb{R}$


DEEPSEEK-V2 MLA: ESSENTIAL INFERENCE FLOW (SIMPLIFIED)
Projection (Simplified)
Current h_t
W_{DKV}
$\mathbb{R}^{d_c}$
c_t^{KV} (Cached Latent)
W_{DQ}
$\mathbb{R}^{d_c'}$
c_t^{Q} (Transient Latent)
Absorbed Scoring & Value
Current h_t
Precomputed $\tilde{W}^{(i)}$
$\tilde{q}_t^{(i)} \in \mathbb{R}^{d_c'}$
$c_t^{KV} \in \mathbb{R}^{d_c}$
absorbed query
$\tilde{q}_t^{(i)} \in \mathbb{R}^{d_c}$
Dot Product
$c_t^{Q} \in \mathbb{R}^{d_c}$
scores
Precomputed $\tilde{W}^{(i)}$
$\mathbb{R}^{d_c'}$
KV Cache, $c_s^{KV} \in \mathbb{R}^{d_c}$
$c_s^{KV} \in \mathbb{R}^{d_c}$
softmax
$\alpha_{ts}^{(i)}$
Attention Weights
$\sum_i$ Average
Weighted Latent
$\bar{c}_t^{(i)} \in \mathbb{R}^{d_c}$
Precomputed $\tilde{W}_O^{(i)}$
$\sum_i$
Output
$out_t \in \mathbb{R}^{d}$
MLA ESSENTIAL PAYOFFS
$n_h \cdot n_h \cdot d_h$
$c_s^{KV} \in \mathbb{R}^{512}$
Small, 512-dim Cached Latents (c_s^{KV})
$\tilde{W}^{(i)}$ & Q
Precomputed matrices fold projections, enabling heads from small latents.
+
total $n_h \cdot d_h$
Massive heads (using original total d_h + a tiny, compared cache.
NO LARGE PER-TOKEN CACHE. LARGE MODEL HEADS, SMALL CACHE. MASSIVE SAVINGS.

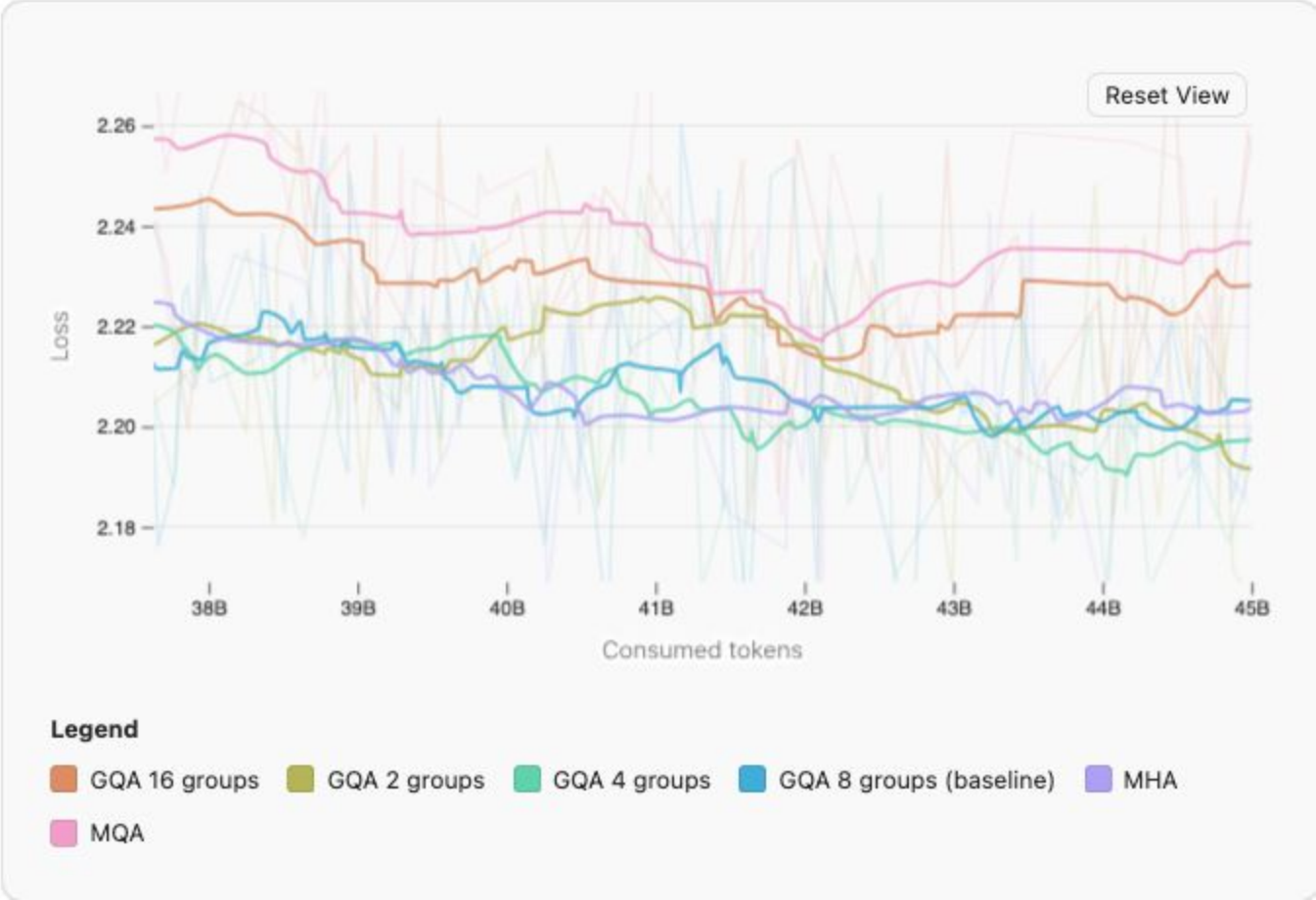
Attention Mechanism	KV Cache Parameters per Token
MHA	$= 2 \times n_{heads} \times n_{layers} \times dim_{head}$
MQA	$= 2 \times 1 \times n_{layers} \times dim_{head}$
GQA	$= 2 \times g \times n_{layers} \times dim_{head}$ (typically g=2,4,8)
MLA	$= 4.5 \times n_{layers} \times dim_{head}$

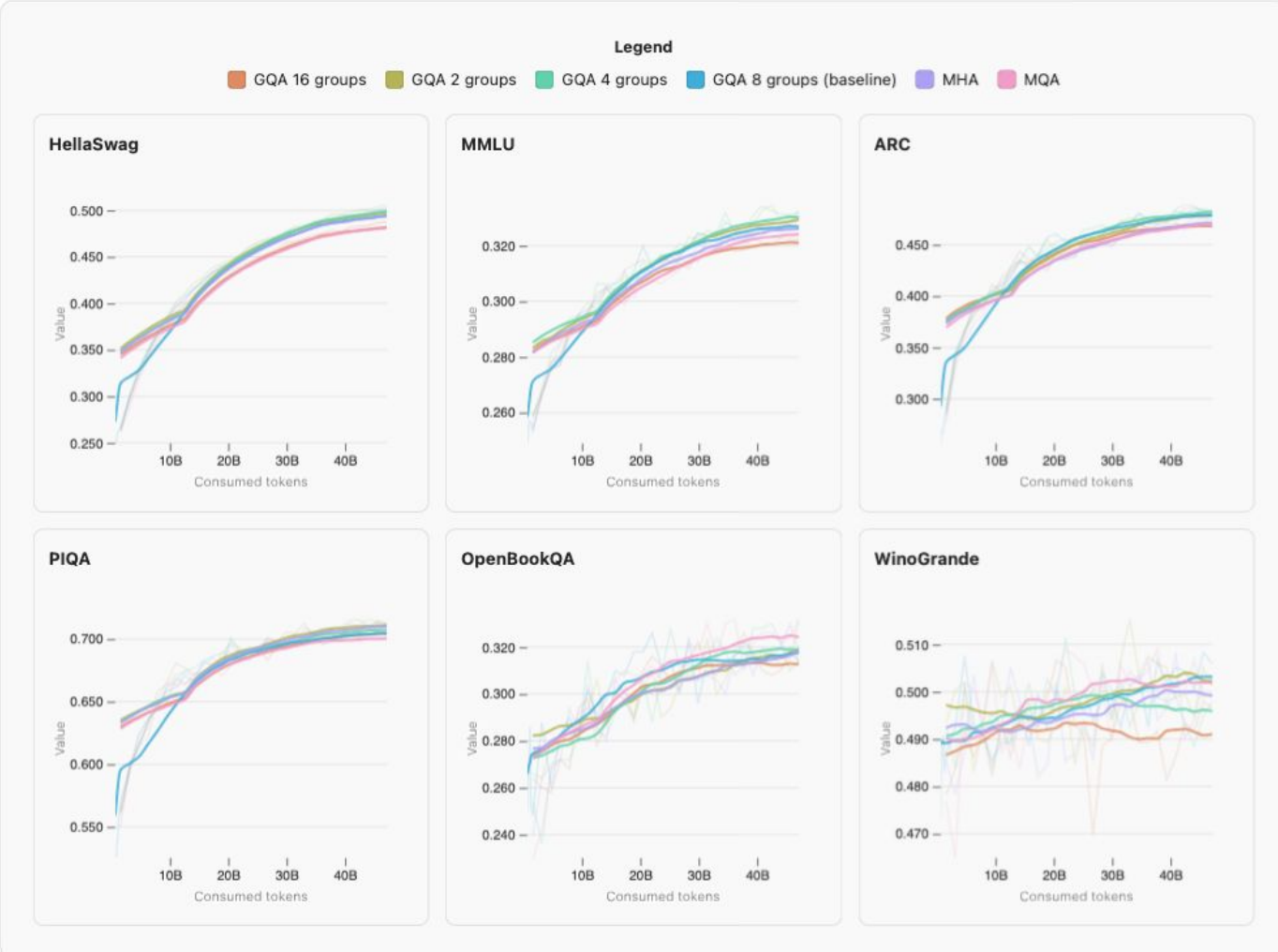
Attention Type	Query Heads	KV Heads	Layers	Parameter Count	Notes
MQA	32	1	16	1.21B	
GQA (ratio 16)	32	2	16	1.21B	
GQA (ratio 8)	32	4	16	1.22B	**Our baseline**
GQA (ratio 4)	32	8	16	1.24B	
GQA (ratio 2)	32	16	15	1.22B	Reduced layers
MHA	32	32	14	1.20B	Reduced layers
GQA (ratio 2)	32	16	16	1.27B	Too large—not abla
MHA	32	32	16	1.34B	Too large—not abla



Legend

GQA 16 groups | GQA 2 groups | GQA 4 groups | GQA 8 groups (baseline) | MHA | MQA

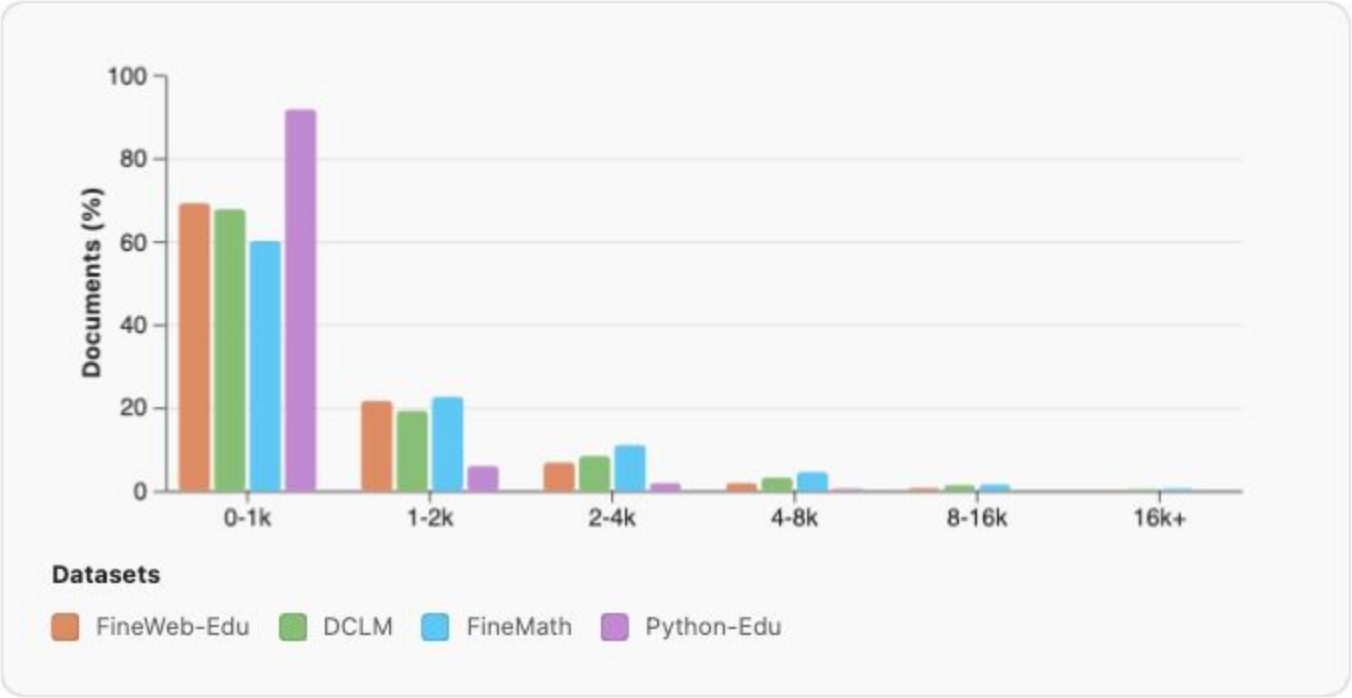

Reset View
2.26
2.24
2.22
2.20
2.18
Loss
38B
39B
40B
41B
42B
43B
44B
45B
Consumed tokens
Legend
GQA 16 groups
GQA 2 groups
GQA 4 groups
GQA 8 groups (baseline)
MHA
MQA


Legend
GQA 16 groups
GQA 2 groups
GQA 4 groups
GQA 8 groups (baseline)
MHA
MQA
HellaSwag
0.500
0.450
0.400
0.350
0.300
0.250
Value
10B
20B
30B
40B
Consumed tokens
MMLU
0.320
0.300
0.280
0.260
Value
10B
20B
30B
40B
Consumed tokens
ARC
0.450
0.400
0.350
0.300
Value
10B
20B
30B
40B
Consumed tokens
PIQA
0.700
0.650
0.600
0.550
Value
10B
20B
30B
40B
Consumed tokens
OpenBookQA
0.320
0.300
0.280
0.260
0.240
Value
10B
20B
30B
40B
Consumed tokens
WinoGrande
0.510
0.500
0.490
0.480
0.470
Value
10B
20B
30B
40B
Consumed tokens

Document Masking

```
File 1: "Recipe for granola bars..." (400 tokens) <EOS>
File 2: "def hello_world()..." (300 tokens) <EOS>
File 3: "Climate change impacts..." (1000 tokens) <EOS>
File 4: "import numpy as np..." (3000 tokens) <EOS>
...

After concatenation and chunking into 4k sequences:
Sequence 1: [File 1] + [File 2] + [File 3] + [partial File 4]
Sequence 2: [rest of File 4] + [File 5] + [File 6] + ...
```

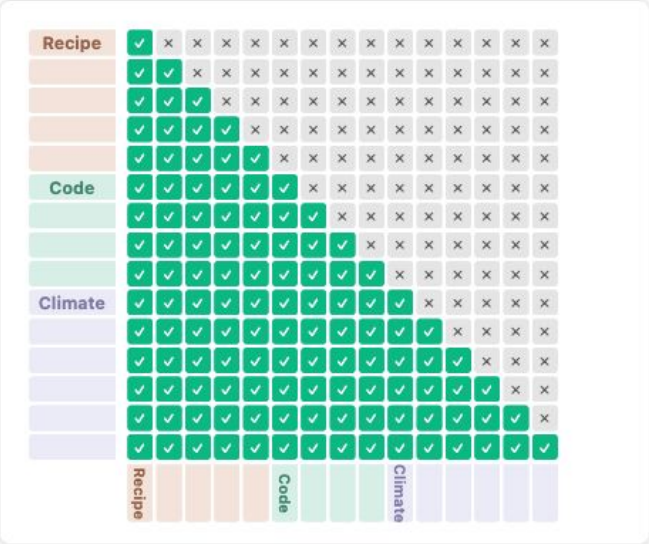

100
80
60
40
20
0
Documents (%)
0-1k
1-2k
2-4k
4-8k
8-16k
16k+
Datasets
FineWeb-Edu
DCLM
FineMath
Python-Edu

Training sequence = concatenated files

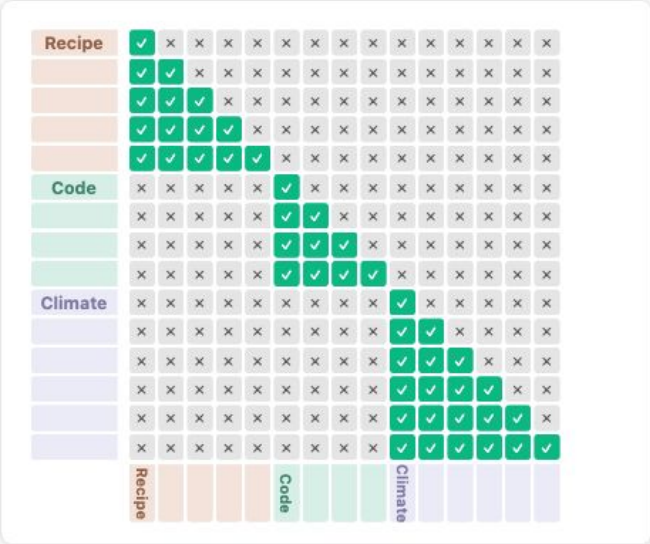
Mix oats with honey <|end_of_text|> def hello(): print <|end_of_text|>

Climate change affects the planet <|end_of_text|>

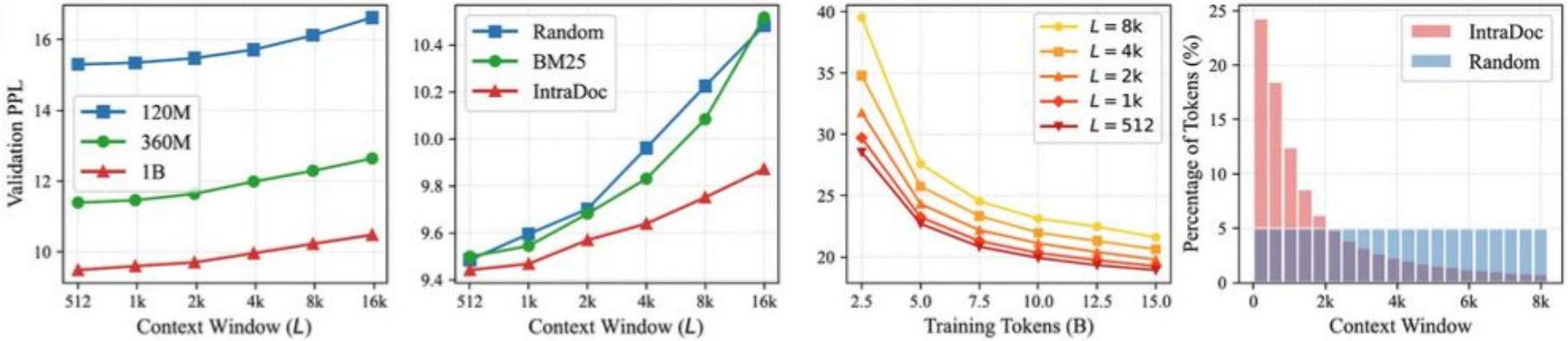
Causal Masking



Intra-Document Masking



✓ Can Attend × Cannot Attend



(a) Validation perplexity of models with varying sizes across different context window sizes.

(b) Validation perplexity of different packing and masking strategies across varying context window sizes.

(c) Validation perplexity of models without positional encoding across varying context window sizes

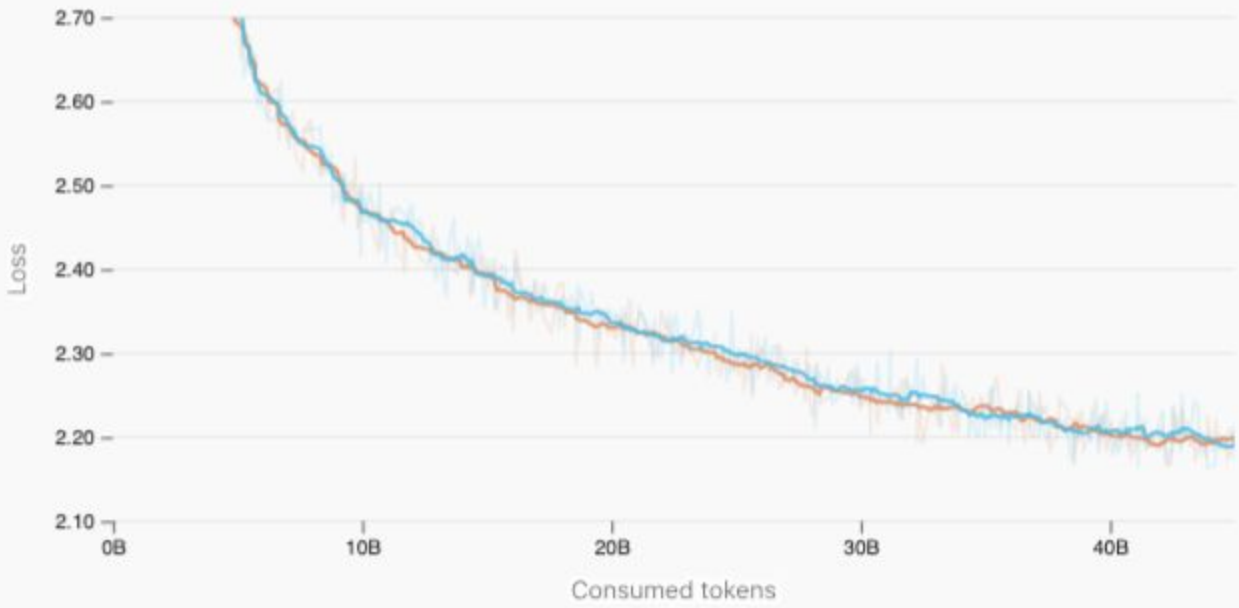
(d) Distribution of context window sizes for IntraDoc and Random methods.

These plots from SkyLadder demonstrate multiple findings: (a) shorter contexts often perform better during pretraining (lower validation perplexity), (b) intra-document masking (IntraDoc) achieves lower perplexity than both random packing (Random) and semantic grouping (BM25), (c) the shorter context advantage holds even without positional encoding, and (d) IntraDoc creates a distribution skewed toward shorter effective context lengths.

Validation perplexity (PPL) is the **exponentiated average negative log-likelihood** that a language model assigns to a held-out text corpus. Formally, for a sequence of tokens $w_1, \ldots, w_N$ drawn from the validation set:

$$\mathrm{PPL} = \exp\left(-\frac{1}{N}\sum_{i=1}^{N}\log p_\theta(w_i \mid w_{<i})\right)$$

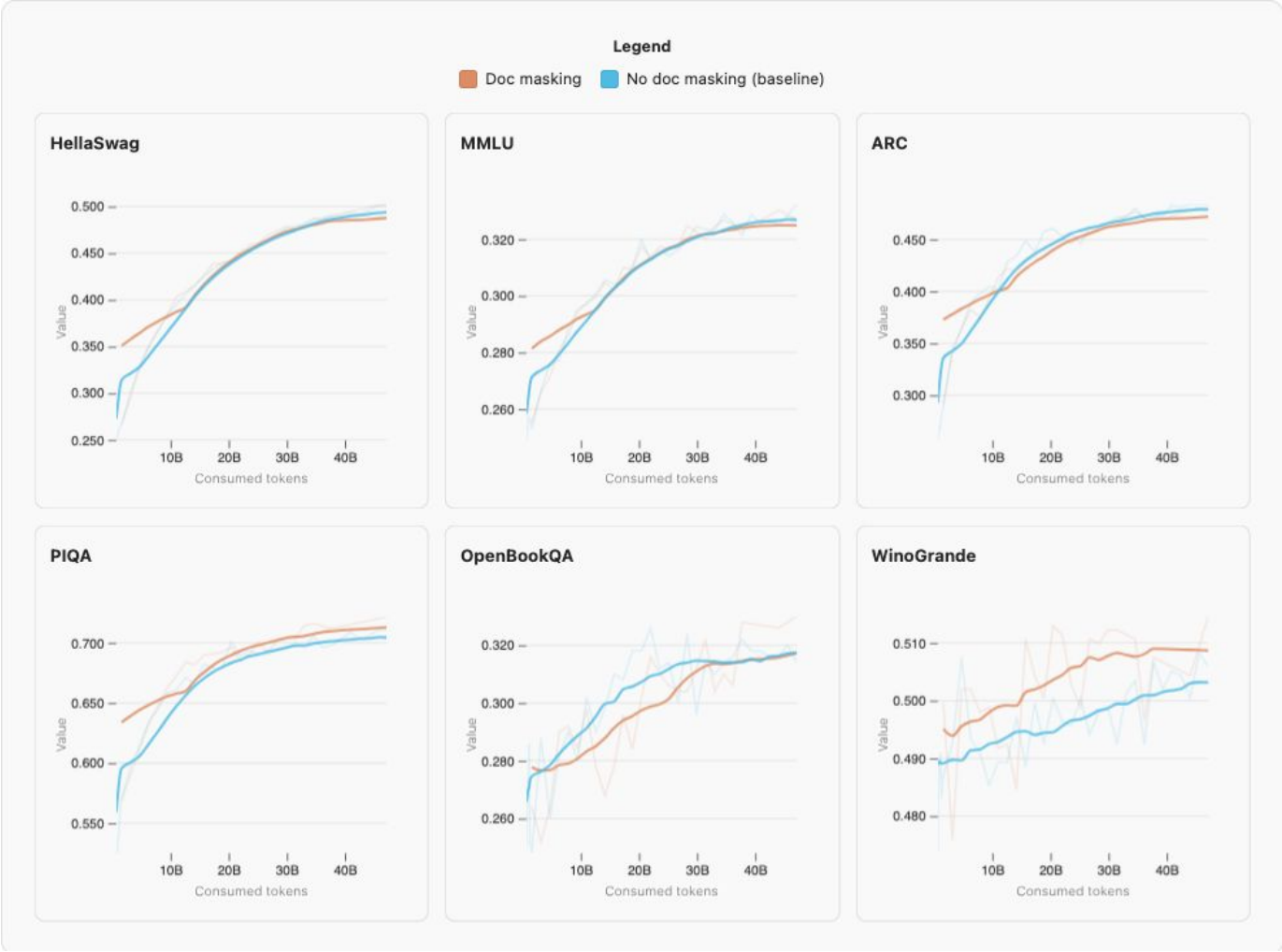
Meta also trained Llama 3 (Grattafiori et al., 2024) with intra-document masking; they found limited impact during short context pretraining but significant benefits for long context extension, where the attention overhead becomes more significant



Legend

Doc masking

No doc masking (baseline)


Legend
Doc masking
No doc masking (baseline)
HellaSwag
0.500
0.450
0.400
0.350
0.300
0.250
Value
10B
20B
30B
40B
Consumed tokens
MMLU
0.320
0.300
0.280
0.260
Value
10B
20B
30B
40B
Consumed tokens
ARC
0.450
0.400
0.350
0.300
Value
10B
20B
30B
40B
Consumed tokens
PIQA
0.700
0.650
0.600
0.550
Value
10B
20B
30B
40B
Consumed tokens
OpenBookQA
0.320
0.300
0.280
0.260
Value
10B
20B
30B
40B
Consumed tokens
WinoGrande
0.510
0.500
0.490
0.480
Value
10B
20B
30B
40B
Consumed tokens